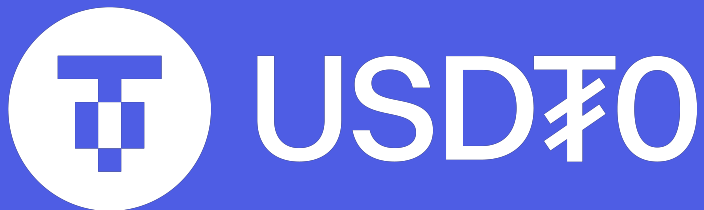


USDT0 Child Token Audit



May 25, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Trust Assumptions and Privileged Roles	5
Low Severity	7
L-01 Missing Support for Contract Signatures	7
L-02 Missing Support for EIP-3009	7
L-03 Redundant Event Emission	8
L-04 Missing ERC-165 Support	8
L-05 Missing Zero-Address Check	9
Notes & Additional Information	9
N-01 Misleading Documentation	9
N-02 The ecrecover Precompile Is Vulnerable to Signature Malleability	10
N-03 Inconsistent Error Messages	10
N-04 Lack of Security Contact	11
N-05 Inconsistent Order Within Contracts	11
N-06 Function Visibility Overly Permissive	12
N-07 Lack of SPDX License Identifier	12
Conclusion	13

Summary

Type	Stablecoins	Total Issues	12 (3 resolved)
Timeline	From 2025-05-05 To 2025-05-09	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	5 (2 resolved)
		Notes & Additional Information	7 (1 resolved)

Scope

OpenZeppelin audited the [Everdawn-Labs/pos-portal](#) repository at commit [6a81fd7](#).

In scope were the following files:

```
contracts
├── child/ChildToken/DappTokens
│   ├── UChildUSDT0.sol
│   └── WithBlockedList.sol
```

System Overview

The new USDT0 implementation is aimed to replace the current USDT deployment on Polygon based on the POS-Portal bridge architecture with a modern, omnichain-capable design. At the core of this upgrade is the [UChildUSDT0](#) contract, which conforms to LayerZero's Omnichain Fungible Token (OFT) standard and extends it by implementing [EIP-7802](#). This enhancement introduces a standardized interface for minting and burning tokens across chains, enabling secure and consistent supply management without relying on wrapped assets or third-party bridges.

USDT0 operates by locking native USDT on Ethereum and minting corresponding USDT0 tokens on other chains such as Polygon. These tokens are fully backed 1:1 and are issued or redeemed based on validated messages transmitted through LayerZero's decentralized messaging network. On Polygon, the [UChildUSDT0](#) contract is responsible for managing user balances, executing mint and burn operations, and enforcing access control via role-based permissions.

Unlike the previous [UChildERC20](#) contract, which was tied to a single bridge mechanism and limited administrative capabilities, [UChildUSDT0](#) serves as a more flexible and secure accounting layer. It receives authenticated instructions from the local OFT contract [OUpgradeable](#), which processes incoming cross-chain messages. This separation of responsibilities, message interpretation by the OFT contract and token accounting by [UChildUSDT0](#), enhances modularity.

This upgrade supports seamless interoperability across supported blockchains, minimizes liquidity fragmentation, and ensures that the same trusted USDT can move freely between ecosystems, using a single, unified standard.

Trust Assumptions and Privileged Roles

The upgraded USDT0 system on Polygon relies on several key trust assumptions and governance mechanisms to maintain security, compliance, and functional integrity. [EIP-7802](#)

establishes that only explicitly authorized contracts may perform cross-chain mint and burn operations. The system enforces this through tightly scoped role-based access control and contract upgrade governance.

Access within the `UChildUSDt0` contract is defined by the following roles:

- **DEFAULT_ADMIN_ROLE** : The primary administrative authority with full control over the contract. This role can mint and burn tokens, destroy funds of blocked users, override blocklist restrictions during transfers, update the associated OFT contract, and modify the token's name and symbol. It also manages all other roles.
- **BLOCK_ROLE** : Responsible for maintaining the blocklist, including adding or removing users. This role ensures compliance with regulatory requirements or internal risk controls by restricting access to the token.
- **DEPOSITOR_ROLE** : Assigned to the OFT contract on the same chain. This is the only entity authorized to call the `crosschainMint` and `crosschainBurn` functions in compliance with EIP-7802. It acts as the endpoint for cross-chain messages interpreted via LayerZero.
- **Proxy Admin**: Controls the upgrade path for the contract logic via the upgradeable proxy. This role must be strictly protected, as it has the ability to introduce or change the behavior of the system entirely, including modifying role permissions or accounting logic.

Furthermore, the trust model assumes that LayerZero's messaging infrastructure functions correctly and securely, meaning that all cross-chain instructions have been authenticated via decentralized oracle and relayer systems. If this layer is compromised, malicious messages could result in unauthorized token issuance or burning.

Similarly, the security of the system depends on the integrity and operational practices of role holders. The `DEFAULT_ADMIN_ROLE` , `BLOCK_ROLE` , and proxy admin are especially sensitive, and should be governed by secure mechanisms such as multisig wallets and time-locked upgrades.

Successful migration from the old `UChildERC20` to the new `UChildUSDt0` contract also assumes that token balances are preserved without loss or disruption, and that the upgrade is transparent to users and integrators while introducing enhanced functionality and security assurances.

Low Severity

L-01 Missing Support for Contract Signatures

The current implementation of [the permit function](#) within the `UChildUSD0` contract is designed to only accept Externally Owned Account (EOA) signatures, utilizing traditional ECDSA verification methods. This approach, while effective for individual users operating EOAs, does not accommodate signatures generated by smart contract accounts. As the ecosystem evolves, with the increasing adoption of smart contract wallets and the emergence of standards like [EIP-7702](#), the inability to process contract signatures could represent a significant limitation.

Consider updating the permit function's signature verification process to include support for both EOA and smart contract account signatures. The OpenZeppelin `SignatureChecker` library provides a unified interface for verifying both ECDSA signatures from EOAs and [EIP-1271](#) compliant signatures from smart contract accounts. This change will make the contract more inclusive and adaptable to the evolving landscape of smart account management.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We tried to stay as close to the current implementation as possible.*

L-02 Missing Support for EIP-3009

The [developer documentation](#) for USD0 indicates support for [EIP-3009](#), which introduces gasless transfers. However, it appears that the current implementation of [the UChildUSD0 contract](#) does not adhere to this standard, as it lacks the `transferWithAuthorization` and `receiveWithAuthorization` functions. This discrepancy between the documentation and the actual contract implementation could lead to confusion and misalignment with user expectations regarding gasless transfer capabilities.

Consider updating the USD0 contract to include the `transferWithAuthorization` and `receiveWithAuthorization` functions, thereby fully supporting EIP-3009 and enhancing the contract's utility and user experience with gasless transfers. Alternatively, consider updating the developer documentation to be inline with the USD0 contract's capabilities.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We have decided to leave out support for EIP-3009 for this deployment.*

L-03 Redundant Event Emission

In `WithBlocklist.sol`, the `addToBlockedList` and `removeFromBlockedList` functions are used to handle the blocklist for users. However, neither function checks whether the new value differs from the existing one before emitting an event. This might lead to redundant event emissions, thereby incurring unnecessary gas costs. More importantly, these redundant events could mislead other actors or monitor systems, suggesting that an address was added or removed from the blocklist whereas it was not.

Consider verifying whether the address is already present in the blocklist before emitting the event.

Update: Acknowledged, not resolved. The Everdawn team stated:

| *We prefer code simplicity and off-chain management. We do not think this should be verified on-chain.*

L-04 Missing ERC-165 Support

The `UChildUSDT0` contract supports [EIP-7802](#) by implementing the `crosschainMint` and `crosschainBurn` functions to manage cross-chain USDT0 token reserves on supported chains. Contrary to EIP-7802's recommendation, `UChildUSDT0` does not implement the [ERC-165](#) interface, lacking the `supportsInterface` function needed to demonstrate support for the interfaces.

Consider implementing ERC-165 by inheriting `IERC165` and defining `supportsInterface` to return `true` for all the supported interfaces, enhancing compatibility with external contracts.

Update: Resolved in [pull request #18](#) at [commit eeaf85e](#).

L-05 Missing Zero-Address Check

The `UChildUSDT0` contract is missing a zero address check when setting a storage variable. Accidentally setting an address variable to address zero might result in an incorrect configuration of the protocol.

More specifically, the `newAdmin` parameter of the `upgradeToUSDT0` function could benefit from a zero-address check to ensure that the new admin is properly set. Accidentally setting this admin to address zero might result in the contract not being able to perform admin capabilities without updating the implementation address.

Consider adding a zero address check to the `newAdmin` parameter in order to avoid accidental misconfigurations.

Update: Resolved in [pull request #18](#) at [commit b6aeae2](#).

Notes & Additional Information

N-01 Misleading Documentation

The [inline documentation](#) for the `executeMetaTransaction` function of the `NativeMetaTransaction` contract inaccurately states that both the `userAddress` and `relayer` address are appended to the calldata for extraction, whereas the implementation only appends the `userAddress`. This discrepancy can lead to misunderstandings about how the contract processes these addresses, especially since `ContextMixin`, which is expected to interact with this data, does not require the `relayer` address.

The primary impact of this misleading documentation is potential confusion among developers and auditors, who may have incorrect expectations about the contract's functionality. Although this does not affect the contract's operation, since only the `userAddress` is needed and correctly handled, the inaccurate comment could mislead individuals reviewing or interacting with the code.

Consider updating the inline documentation to accurately describe the actual implementation. This correction will align the documentation with the code's functionality and ensure clarity for all parties reviewing the contract.

Update: Acknowledged, not resolved.

N-02 The `ecrecover` Precompile Is Vulnerable to Signature Malleability

The [use of `ecrecover`](#) for signature verification in the permit function exposes the system to potential signature malleability attacks, where a signature could be altered without access to the private key. However, the implementation mitigates the risk of replay attacks by incorporating nonces, ensuring that each action is unique and cannot be executed more than once. While this significantly reduces the potential impact of such vulnerabilities, it does not eliminate the underlying risk associated with signature malleability.

The primary impact of relying on `ecrecover` is the theoretical risk of signature malleability, which could allow attackers to manipulate signatures in a way that the original signer did not intend. However, in this case, the use of nonces effectively prevents replay attacks.

To enhance security and address the signature malleability issue more comprehensively, consider replacing the use of `ecrecover` with OpenZeppelin's `SignatureChecker` library. This library offers a robust solution for signature verification, supporting both ECDSA signatures from EOAs and [EIP-1271](#) signatures from smart contracts. Adopting this library would not only mitigate the risk of signature malleability but also align the contract with best practices regarding support for smart account wallets.

Update: Acknowledged, not resolved.

N-03 Inconsistent Error Messages

The [UChildUSDt0 contract](#) is inconsistent in the naming conventions used within the error messages. Specifically, some error messages refer to `TetherToken`, while others use `UChildUSDt0`. This inconsistency can lead to confusion for developers and users interacting with the contract, as it may not be immediately clear that both names refer to the same entity within the contracts.

Consider reviewing all error messages within the contract to identify any discrepancies in naming conventions and updating the error messages to reflect a consistent naming approach.

Update: Resolved in [pull request #21](#) at [commit 3de98b1](#).

N-04 Lack of Security Contact

Providing a specific security contact (such as an email or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

More specifically, the [UChildUSDT0 contract](#) does not have a security contact.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the [@custom:security-contact](#) convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: *Acknowledged, not resolved.*

N-05 Inconsistent Order Within Contracts

Both the [UChildUSDT0](#) and [WithBlockedList](#) contracts deviate from the Solidity Style Guide due to having an inconsistent ordering of functions and events:

- The [UChildUSDT0](#) contract is mixing [external](#) with [public](#) and [internal](#) functions.
- Both the [UChildUSDT0](#) and [WithBlockedList](#) contracts define events at the bottom of the contract.

To improve the project's overall legibility, consider standardizing ordering throughout the codebase as recommended by the [Solidity Style Guide \(Order of functions\)](#).

Update: *Acknowledged, not resolved.*

N-06 Function Visibility Overly Permissive

Throughout the codebase, multiple functions with unnecessarily permissive visibility were identified. The following functions with `public` visibility could be limited to `external`:

- In `UChildUSDT0.sol`, the `upgradeToUSDT0`, `transferFrom`, `mint`, `redeem`, and `destroyBlockedFunds` functions
- In `WithBlockedList.sol`, the `addToBlockedList` and `removeFromBlockedList` functions

To better convey the intended use of functions and to potentially realize some additional gas savings, consider changing a function's visibility to be only as permissive as required.

Update: *Acknowledged, not resolved.*

N-07 Lack of SPDX License Identifier

The `UChildUSDT0.sol` file lacks an SPDX license identifier.

To avoid legal issues regarding copyright and follow best practices, consider adding SPDX license identifiers to files as suggested by the [Solidity documentation](#).

Update: *Acknowledged, not resolved.*

Conclusion

OpenZeppelin audited the new USDT0 architecture upgrade for the USDT token on Polygon, which adopts LayerZero's OFT standard and introduces support for EIP-7802. This upgrade transitions the system away from the legacy POS-Portal bridge, enabling standardized and secure cross-chain mint and burn operations. The audit focused on the [UChildUSDT0](#) contract and its role in managing accounting, access control, and EIP-7802 compliance.

The review identified a few areas for improvement, including missing support for standards such as EIP-1271, EIP-165, and EIP-3009, an issue involving the blocklist logic, and minor suggestions to improve code clarity and robustness. The Everdawn Labs team was responsive and collaborative throughout the audit process, promptly addressing questions and providing clarifications where needed.